Pow 算子测试设计说明

1. 测试设计思路

本测试采用 **example 风格端到端测试**，目标不是只验证接口能否调用成功，而是同时覆盖：

- TensorScalar
- InplaceTensorScalar
- ScalarTensor
- TensorTensor
- InplaceTensorTensor
- Exp2
- InplaceExp2

其中重点放在 **op_api/aclnn_pow.cpp**，因为该文件负责：

- TensorScalar / ScalarTensor 的参数校验
- 特殊指数分支
- dtype 推导与类型提升
- 部分错误分支与原地接口处理

测试策略分为两类：

1. **正向执行用例**
    - 真实调用 GetWorkspaceSize + Execute
    - 回拷结果
    - CPU 端用 std::pow / 2^x 计算期望值
    - 做逐元素结果比对
2. **异常输入用例**
    - 用于覆盖 GetWorkspaceSize 中的参数校验分支
    - 验证返回值应为非成功状态

---

2. 用例设计目标

2.1 API 覆盖目标

本测试覆盖了 7 个 API：

- aclnnPowTensorScalar
- aclnnInplacePowTensorScalar
- aclnnPowScalarTensor
- aclnnPowTensorTensor
- aclnnInplacePowTensorTensor
- aclnnExp2
- aclnnInplaceExp2

其中：

- TensorScalar / ScalarTensor / InplaceTensorScalar 主要覆盖 aclnn_pow.cpp
- TensorTensor / InplaceTensorTensor 主要覆盖 aclnn_pow_tensor_tensor.cpp
- Exp2 / InplaceExp2 用于补充 Pow 家族公共调度路径

2.2 dtype 覆盖目标

TensorTensor

覆盖了题目要求的 7 种 dtype：

- FLOAT

- FLOAT16
- BF16
- UINT8
- INT8
- INT16
- INT32

TensorScalar / ScalarTensor

覆盖了多种 scalar / tensor 组合：

- FLOAT
- FLOAT16
- BF16
- UINT8
- INT8
- INT16
- INT32
- INT64
- DOUBLE（主要用于非法输入）

设计依据是优先覆盖：

- 特殊指数处理
- 整数/浮点 exponent 校验
- 溢出检查分支
- dtype mismatch / unsupported dtype 分支

2.3 shape / format 覆盖目标

shape

覆盖了：

- 正常同 shape
- 广播 shape（TensorTensor）
- 空 Tensor：{0, 4}
- shape mismatch 负例

format

覆盖了：

- ACL_FORMAT_ND
- ACL_FORMAT_NCHW

并设计了 format mismatch 负例，用于覆盖 CheckFormat 分支。

2.4 边界条件覆盖目标

覆盖的边界条件包括：

- exponent = 0
- exponent = 1
- exponent = 0.5
- exponent = 3
- exponent = 4
- exponent = -1

- base = 1（触发 ScalarTensor 的 fill-one 路径）
- 整数底数 + 负整数指数（非法）
- nullptr 输入
- 输出 shape / dtype 不匹配
- unsupported dtype（如 DOUBLE）

---

3. 结果验证策略

所有正向用例都进行了数值校验：

- TensorScalar：std::pow(self[i], exponent)
- ScalarTensor：std::pow(base, exponent[i])
- TensorTensor：std::pow(self[i], exponent[i])
- Exp2：std::pow(2.0, self[i])

比较策略：

- 整型：精确比较
- FLOAT：atol=rtol=1e-4
- FLOAT16：atol=rtol=5e-2
- BF16：atol=rtol=8e-2

同时在 CPU 端先按目标 dtype 做量化，避免出现“参考值精度高于设备输出格式”造成的伪失败。

---

4. 覆盖率分析

4.1 已覆盖较好的部分

op_api/aclnn_pow_tensor_tensor.cpp

- 行覆盖较高
- TensorTensor / InplaceTensorTensor 已基本覆盖主要执行路径

op_host/pow_tensor_tensor_tiling_arch35.cpp

- 覆盖率很高
- 说明 TensorTensor 的多 dtype 分发基本已打到

op_api/aclnn_pow.cpp

重点覆盖了以下逻辑：

- TensorScalar / ScalarTensor 正常入口
- InplaceTensorScalar 入口
- 特殊指数分支（0 / 1 / 0.5 / 3 / 4 / -1）
- 整数底数负指数非法分支
- nullptr / shape mismatch / dtype mismatch / unsupported dtype
- ScalarTensor(base=1) 触发 BuildPowScalarTensorFillOne

4.2 当前仍未充分覆盖的部分

根据 .gcov 分析，未覆盖主要集中在以下几类：

1）complex 相关分支

如：

- InnerTypeToComplexType
- CombineCategoriesWithComplex 中的 complex 分支

原因：当前比赛题目与运行环境主要面向 Pow 已注册 dtype,测试未引入 complex 类型；

同时 example 测试也不适合扩展到未注册复杂路径。

2）IsRegBase() 之外的另一套 dtype 推导路径

如：

- InferTensorScalarDtype 中 196-205 行
- InferScalarTensorDtype 中 218-227 行
- isFloatType

原因： 当前环境下 IsRegBase() 路径始终成立，导致非 regbase 分支无法进入，因此这部分难以通过测试侧进一步提升。

3）部分 BF16 / AICPU / SoC 特定路径

如：

- CheckDtypeValid 中 “当前 soc 不支持 BF16”
- IsPowAiCpuOn910B 的部分条件分支

原因： 这些分支依赖具体 SoC 平台能力与运行环境，CPU 模拟器下无法自由切换平台条件。

4）部分 overflow / promoteType 极端分支

如：

- exponent 转换到 promote type 的更多溢出形式
- complex overflow 分支

原因：部分分支依赖更特殊的 scalar 类型组合或内部路径选择，已通过整数负指数、dtype mismatch、unsupported dtype 等覆盖了主要有效错误路径，但极端组合仍未完全进入。

---

5. 当前测试的有效性分析

本测试的优化重点不是盲目增加 case 数量，而是围绕 aclnn_pow.cpp 的真实薄弱点做增强，主要体现在：

- 增加 ScalarTensor(base=1)，直接覆盖 fill-one 分支
- 增加 TensorScalar 整数底数负指数负例，覆盖专门的 exponent 校验逻辑
- 增加多种整数 scalar 类型，补充 exponent overflow 检查分支
- 增加 TensorScalar / ScalarTensor 各自独立的 shape / dtype / nullptr 校验分支

因此，本测试不仅验证了数值正确性，也较真实地覆盖了 Pow 算子的参数校验与分派逻辑。

---

6. 总结

本测试遵循以下原则：

- **以 API 家族为主线：**保证 7 个 API 均被覆盖
- **以 aclnn_pow.cpp 为优化重点：**针对 TensorScalar / ScalarTensor 的关键分支定向补强
- **以真实执行为基础：**正向用例全部执行并校验结果
- **以有效负例覆盖错误路径：**重点覆盖 nullptr、shape mismatch、dtype mismatch、unsupported dtype、非法指数等

从覆盖率结果看，TensorTensor 相关文件已覆盖较充分；当前主要瓶颈集中在 aclnn_pow.cpp 中与平台条件、complex 路径、非 regbase 路径相关的未达分支，这些分

支受运行环境限制，测试侧可提升空间相对有限。